

grpc在Windows上的使用MinGW安装与使用

安装

首先将其git clone下来，放到本地,我是用gitee加速了

```
git clone -b v1.50.0 https://gitee.com/feuyeux/grpc.git
```

我克隆的不是最新版本，因为需要换源，有些源找不到，就用了老一点的版本

然后打开grpc目录，找到.gitmodules这个文件，这里面是子模块的源，把里面的内容换成下面这个

```
[submodule "third_party/abseil-cpp"]
  path = third_party/abseil-cpp
  url = https://gitee.com/suuair/abseil-cpp.git
[submodule "third_party/benchmark"]
  path = third_party/benchmark
  url = https://gitee.com/chahan/benchmark.git
[submodule "third_party/bloaty"]
  path = third_party/bloaty
  url = https://gitee.com/GgBoy_ssz/bloaty.git
[submodule "third_party/boringssl-with-bazel"]
  path = third_party/boringssl-with-bazel
  url = https://gitee.com/GgBoy_ssz/boringssl.git
[submodule "third_party/cares/cares"]
  path = third_party/cares/cares
  url = https://gitee.com/RicLee/c-ares.git
[submodule "third_party/envoy-api"]
  path = third_party/envoy-api
  url = https://gitee.com/RicLee/data-plane-api.git
[submodule "third_party/googleapis"]
  path = third_party/googleapis
  url = https://gitee.com/GgBoy_ssz/googleapis.git
[submodule "third_party/googletest"]
  path = third_party/googletest
  url = https://gitee.com/bosspoi/googletest.git
[submodule "third_party/libuv"]
  path = third_party/libuv
  url = https://gitee.com/RicLee/libuv.git
[submodule "third_party/opencensus-proto"]
  path = third_party/opencensus-proto
  url = https://gitee.com/RicLee/opencensus-proto.git
```

```
[submodule "third_party/opentelemetry"]
    path = third_party/opentelemetry
    url = https://gitee.com/EBServerStudy/opentelemetry-proto.git
[submodule "third_party/protobuf"]
    path = third_party/protobuf
    url = https://gitee.com/EBServerStudy/protobuf.git
[submodule "third_party/protoc-gen-validate"]
    path = third_party/protoc-gen-validate
    url = https://gitee.com/arzhe/protoc-gen-validate.git
[submodule "third_party/re2"]
    path = third_party/re2
    url = https://gitee.com/GgBoy_ssz/re2.git
[submodule "third_party/xds"]
    path = third_party/xds
    url = https://gitee.com/EBServerStudy/xds.git
[submodule "third_party/zlib"]
    path = third_party/zlib
    url = https://gitee.com/RicLee/zlib.git
# When using CMake to build, the zlib submodule ends up with a
# generated file that makes Git consider the submodule dirty. This
# state can be ignored for day-to-day development on gRPC.
ignore = dirty
```

替换后在终端grpc目录下运行下面两条命令

```
git submodule sync
git submodule update --init
```

这样就是成功了

```
D:\Study\practice\test\grpc>git submodule update --init
Submodule 'third_party/abseil-cpp' (https://gitee.com/suair/abseil-cpp.git) registered for path 'third_party/abseil-cpp'
Submodule 'third_party/benchmark' (https://gitee.com/chahan/benchmark.git) registered for path 'third_party/benchmark'
Submodule 'third_party/bloaty' (https://gitee.com/GgBoy_ssz/bloaty.git) registered for path 'third_party/bloaty'
Submodule 'third_party/boringssl-with-bazel' (https://gitee.com/GgBoy_ssz/boringssl.git) registered for path 'third_party/boringssl-with-bazel'
Submodule 'third_party/cares/cares' (https://gitee.com/RicLee/c-ares.git) registered for path 'third_party/cares/cares'
Submodule 'third_party/envoy-api' (https://gitee.com/RicLee/data-plane-api.git) registered for path 'third_party/envoy-api'
Submodule 'third_party/googleapis' (https://gitee.com/GgBoy_ssz/googleapis.git) registered for path 'third_party/googleapis'
Submodule 'third_party/googletest' (https://gitee.com/bosspoi/googletest.git) registered for path 'third_party/googletest'
Submodule 'third_party/libuv' (https://gitee.com/RicLee/libuv.git) registered for path 'third_party/libuv'
Submodule 'third_party/opencensus-proto' (https://gitee.com/RicLee/opencensus-proto.git) registered for path 'third_party/opencensus-proto'
Submodule 'third_party/opentelemetry' (https://gitee.com/EBServerStudy/opentelemetry-proto.git) registered for path 'third_party/opentelemetry'
Submodule 'third_party/protobuf' (https://gitee.com/EBServerStudy/protobuf.git) registered for path 'third_party/protobuf'
Submodule 'third_party/re2' (https://gitee.com/GgBoy_ssz/re2.git) registered for path 'third_party/re2'
Submodule 'third_party/xds' (https://gitee.com/EBServerStudy/xds.git) registered for path 'third_party/xds'
Submodule 'third_party/zlib' (https://gitee.com/RicLee/zlib.git) registered for path 'third_party/zlib'
Cloning into 'E:/Study/practice/test/grpc/third_party/abseil-cpp'...
Cloning into 'E:/Study/practice/test/grpc/third_party/benchmark'...
Cloning into 'E:/Study/practice/test/grpc/third_party/bloaty'...
Cloning into 'E:/Study/practice/test/grpc/third_party/boringssl-with-bazel'...
Cloning into 'E:/Study/practice/test/grpc/third_party/cares/cares'...
Cloning into 'E:/Study/practice/test/grpc/third_party/envoy-api'...
Cloning into 'E:/Study/practice/test/grpc/third_party/googleapis'...
Cloning into 'E:/Study/practice/test/grpc/third_party/googletest'...
Cloning into 'E:/Study/practice/test/grpc/third_party/libuv'...
Cloning into 'E:/Study/practice/test/grpc/third_party/opencensus-proto'...
Cloning into 'E:/Study/practice/test/grpc/third_party/opentelemetry'...
Cloning into 'E:/Study/practice/test/grpc/third_party/protobuf'...
Cloning into 'E:/Study/practice/test/grpc/third_party/re2'...
Cloning into 'E:/Study/practice/test/grpc/third_party/xds'...
Cloning into 'E:/Study/practice/test/grpc/third_party/zlib'...
Submodule path 'third_party/abseil-cpp': checked out 273292d1cfc0a94a65082ee350509af1d113344d
Submodule path 'third_party/benchmark': checked out 361e8d1cfe0c6c36d30b39f1b61302ece5507320
Submodule path 'third_party/bloaty': checked out 60209eb1ccc34d5deeff002d1b7f37545204f7f2
Submodule path 'third_party/boringssl-with-bazel': checked out b9232f9e27e5668bc0414879dcddeb2a59ea75f2
Submodule path 'third_party/cares/cares': checked out 6654436a307a5a686b008c1d4c93b0085da6e6d8
Submodule path 'third_party/envoy-api': checked out bf6154e432bbd5e6f64032993206e66b6116f2bd
Submodule path 'third_party/googleapis': checked out 2f9af297c84c55c8b871ba4495e01ade42476c92
Submodule path 'third_party/googletest': checked out 0e402173c97aea7a00749e825b194bfede4f2e45
Submodule path 'third_party/libuv': checked out 02a9e1be252b623ee032a3137c0b0c94afbe6809
Submodule path 'third_party/opencensus-proto': checked out 4aa53e15cbf1a47bc9087e6cfda214c1eea4e89
Submodule path 'third_party/opentelemetry': checked out 60fa8754d8390b5c55949a8c68dcfd7ab5c2395df
Submodule path 'third_party/protobuf': checked out 24487dd1045c7f3d64a21f38a3f0c06cc4cf2edb
Submodule path 'third_party/re2': checked out 8e08f47b11b413302749c0d8b17alc94777495d5
Submodule path 'third_party/xds': checked out cb28da3451f158a947dfc45090fe92b07b243bc1
Submodule path 'third_party/zlib': checked out 21767c654d31d2dcccde4330529775c6c5fd5389
```

在 grpc/cmake 下 创建一个 build 目录用于生成 makefile , 然后在 build 目录下运行下面的命令 ,-DCMAKE_INSTALL_PREFIX指定安装目录

```
cmake ../../ -G "MinGW Makefiles" -DCMAKE_INSTALL_PREFIX=PREFIX=D:/Clion/test
make -j6
make install -j6
```

成功后在安装目录下会看到一个grpc目录，结构如下

名称	修改日期	类型	:
bin	2024/12/26 14:22	文件夹	
include	2024/12/26 14:22	文件夹	
lib	2024/12/26 14:22	文件夹	
share	2024/12/26 14:22	文件夹	

bin目录中是一些可执行程序，如portobuf解析的protoc， grpc的protobuf的cpp生成插件grpc_cpp_plugin， 这些后面都要用， lib是一些外部库文件， include是一些头文件。

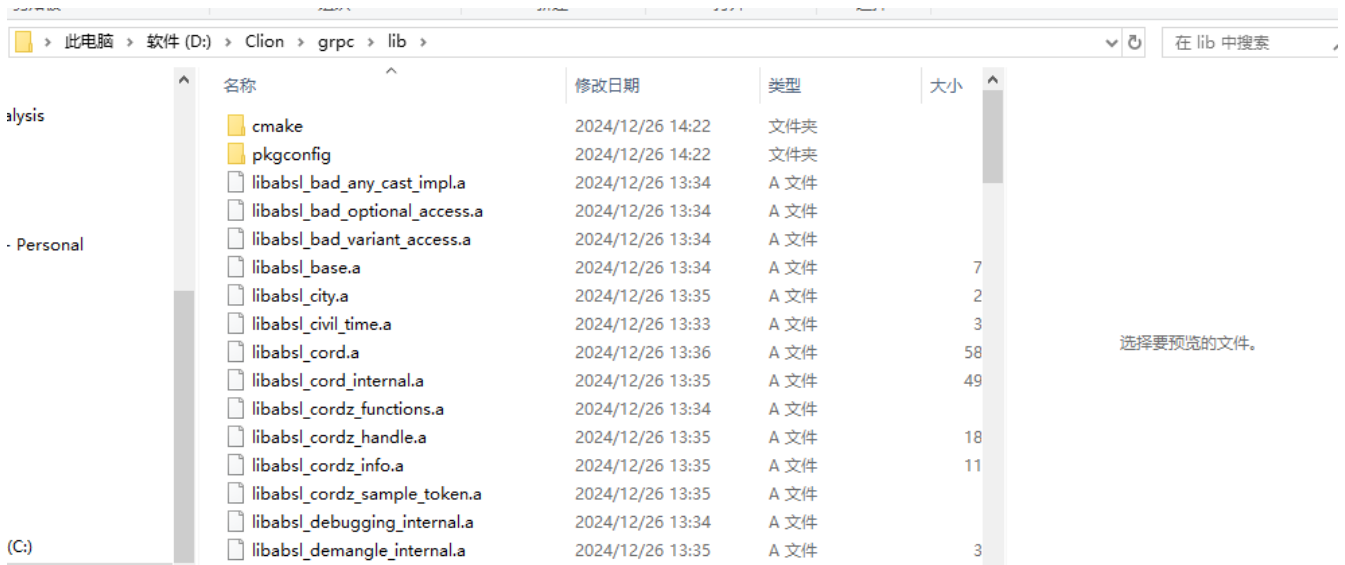
把grpc安装的根目录， bin， lib加到环境变量中(不加也可以， 不过后续写cmake需要自己指定下目录)

名称	修改日期	类型	大小
acountry.exe	2024/12/26 13:34	应用程序	248 KB
adig.exe	2024/12/26 13:34	应用程序	187 KB
ahost.exe	2024/12/26 13:34	应用程序	206 KB
grpc_cpp_plugin.exe	2024/12/26 13:39	应用程序	9,735 KB
grpc_csharp_plugin.exe	2024/12/26 13:39	应用程序	9,756 KB
grpc_node_plugin.exe	2024/12/26 13:39	应用程序	9,512 KB
grpc_objective_c_plugin.exe	2024/12/26 13:39	应用程序	9,696 KB
grpc_php_plugin.exe	2024/12/26 13:39	应用程序	9,808 KB
grpc_python_plugin.exe	2024/12/26 13:39	应用程序	9,778 KB
grpc_ruby_plugin.exe	2024/12/26 13:39	应用程序	9,467 KB
libzlib.dll	2024/12/26 13:34	应用程序扩展	198 KB
protoc.exe	2024/12/26 13:38	应用程序	17,535 KB

(D:) > Clion > grpc > include

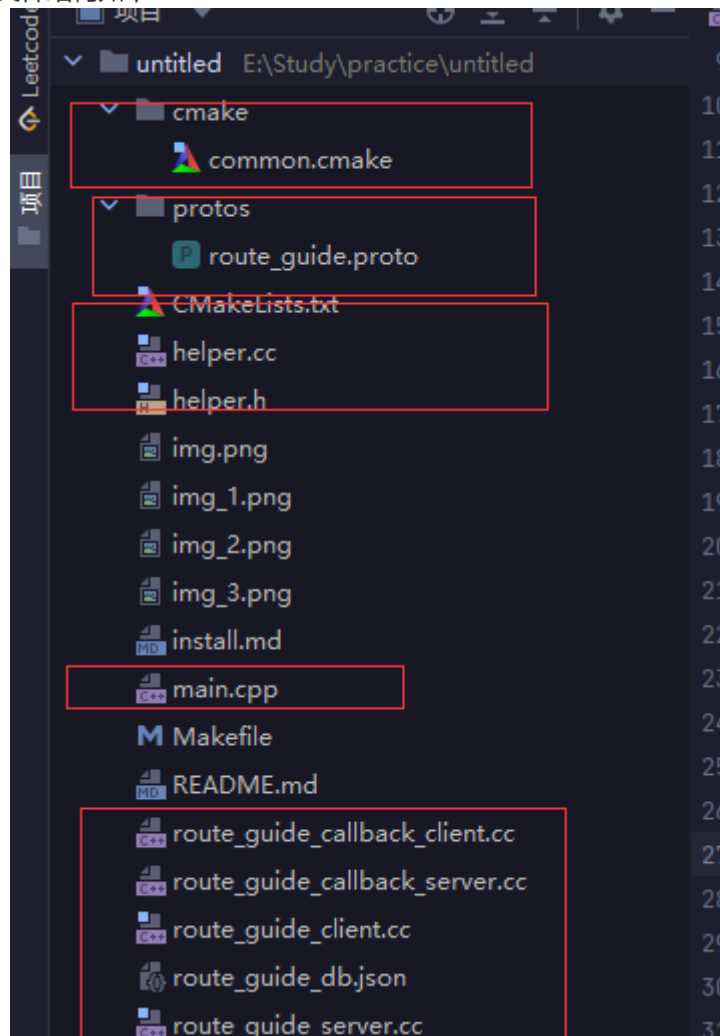
名称	修改日期	类型	大小
absl	2024/12/26 14:22	文件夹	
google	2024/12/26 14:22	文件夹	
grpc	2024/12/26 14:22	文件夹	
grpc++	2024/12/26 14:22	文件夹	
grpcpp	2024/12/26 14:22	文件夹	
re2	2024/12/26 14:22	文件夹	
ares.h	2024/12/26 13:21	H 文件	27 k
ares_build.h	2024/12/26 13:25	H 文件	1 k
ares_dns.h	2024/12/26 13:21	H 文件	6 k
ares_rules.h	2024/12/26 13:21	H 文件	5 k
ares_version.h	2024/12/26 13:21	H 文件	1 k
zconf.h	2024/12/26 13:32	H 文件	17 k
zlib.h	2024/12/26 13:21	H 文件	97 k

选择要预览的文件。



使用

这里我们打开之前 git 下来的 grpc 目录，进入 `grpc/examples/cpp`，我们需要的是里面的 `protos/route_guide.proto`, `cmake/common.cmake`, `route_guide/CMakeLists.txt`, `route_guide` 下的所有 `.c`, `.cc`, `.json` 文件，将他们拷贝到一个新目录下进行测试，我这里使用的是 Clion，当然用 `cmd`，`vscode` 等其他工具也可以，只要有对应的工具链就可以，步骤大差不差。文件结构如下：



cmakelist在官方的基础上也要做一些改变

```
# Copyright 2018 gRPC authors.
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
#
# cmake build file for C++ route_guide example.
# Assumes protobuf and gRPC have been installed using cmake.
# See cmake_externalproject/CMakeLists.txt for all-in-one cmake build
# that automatically builds all the dependencies before building route_guide.

cmake_minimum_required(VERSION 3.5.1)

project(RouteGuide C CXX)

include(cmake/common.cmake)

# Proto file
get_filename_component(rg_proto "../protos/route_guide.proto" ABSOLUTE)
get_filename_component(rg_proto_path "${rg_proto}" PATH)

# Generated sources
set(rg_proto_srcs "${CMAKE_CURRENT_BINARY_DIR}/route_guide.pb.cc")
set(rg_proto_hdrs "${CMAKE_CURRENT_BINARY_DIR}/route_guide.pb.h")
set(rg_grpc_srcs "${CMAKE_CURRENT_BINARY_DIR}/route_guide.grpc.pb.cc")
set(rg_grpc_hdrs "${CMAKE_CURRENT_BINARY_DIR}/route_guide.grpc.pb.h")
add_custom_command(
    OUTPUT "${rg_proto_srcs}" "${rg_proto_hdrs}" "${rg_grpc_srcs}" "${rg_grpc_hdrs}"
    COMMAND ${_PROTOBUF_PROTOC}
    ARGS --grpc_out "${CMAKE_CURRENT_BINARY_DIR}"
    --cpp_out "${CMAKE_CURRENT_BINARY_DIR}"
    -I "${rg_proto_path}"
    --plugin=protoc-gen-grpc= "${_GRPC_CPP_PLUGIN_EXECUTABLE}"
    "${rg_proto}"
    DEPENDS "${rg_proto}")

# Include generated *.pb.h files
include_directories("${CMAKE_CURRENT_BINARY_DIR}")

# rg_grpc_proto
add_library(rg_grpc_proto
    ${rg_grpc_srcs}
    ${rg_grpc_hdrs}
    ${rg_proto_srcs}
    ${rg_proto_hdrs})
target_link_libraries(rg_grpc_proto
```

```

ws2_32
${_REFLECTION}
${_GRPC_GRPCPP}
${_PROTOBUF_LIBPROTOBUF})

# route_guide_helper
add_library(route_guide_helper
    "helper.h"
    "helper.cc")
target_link_libraries(route_guide_helper
    ws2_32
    rg_grpc_proto
    ${_REFLECTION}
    ${_GRPC_GRPCPP}
    ${_PROTOBUF_LIBPROTOBUF})

# Targets route_guide_(client|server)
foreach (_target
    route_guide_client route_guide_server)
    add_executable(${_target}
        "${_target}.cc")
    target_link_libraries(${_target}
        rg_grpc_proto
        route_guide_helper
        ${_REFLECTION}
        ${_GRPC_GRPCPP}
        ${_PROTOBUF_LIBPROTOBUF})
endforeach ()

```

主要的两处改变

文件路径

```

project(RouteGuide C CXX)

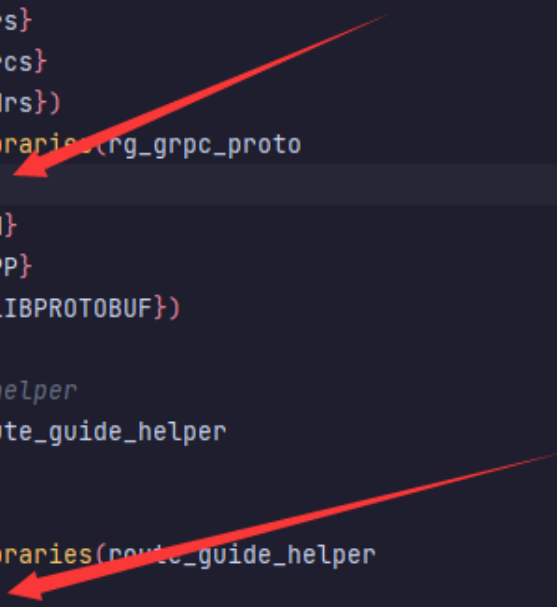
include(cmake/common.cmake)

# Proto file
get_filename_component(rg_proto "protos/route_guide.proto" ABSOLUTE)
get_filename_component(rg_proto_path "${rg_proto}" PATH)

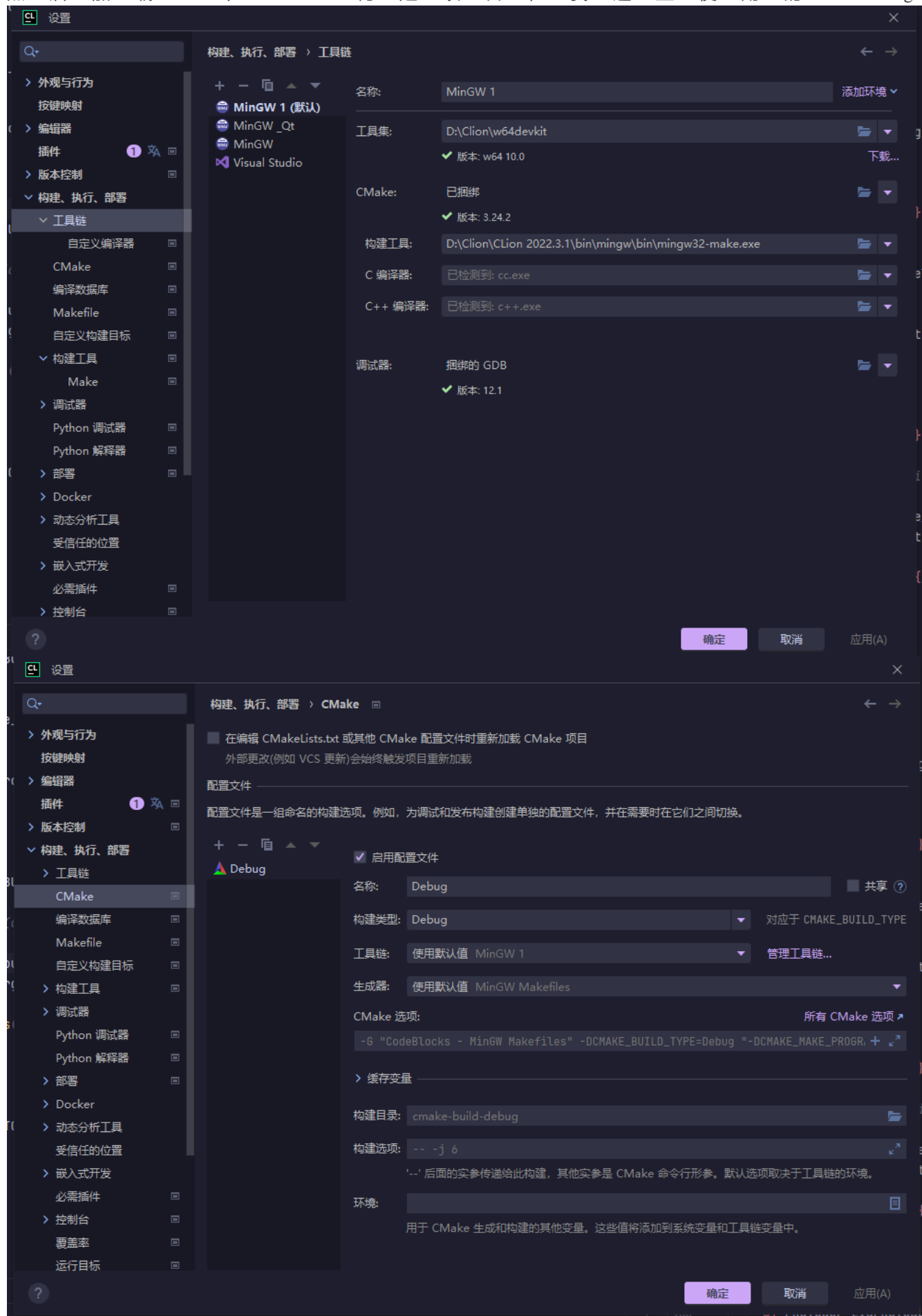
```

Windows上的网路库

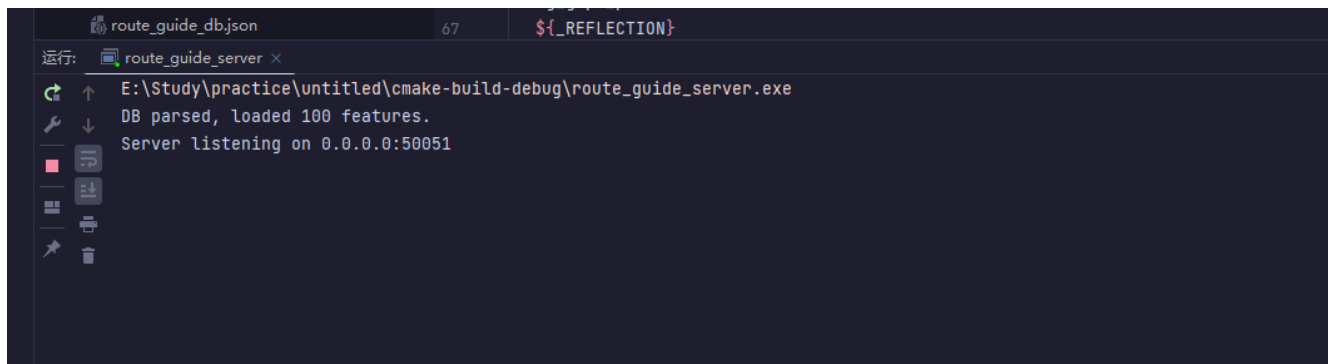
```
46 include_directories("${CMAKE_CURRENT_BINARY_DIR}")
47
48 # rg_grpc_proto
49 add_library(rg_grpc_proto
50     ${rg_grpc_srcs}
51     ${rg_grpc_hdrs}
52     ${rg_proto_srcs}
53     ${rg_proto_hdrs})
54 target_link_libraries(rg_grpc_proto
55     ws2_32
56     ${_REFLECTION}
57     ${_GRPC_GRPCPP}
58     ${_PROTOBUF_LIBPROTOBUF})
59
60 # route_guide_helper
61 add_library(route_guide_helper
62     "helper.h"
63     "helper.cc")
64 target_link_libraries(route_guide_helper
65     ws2_32
66     rg_grpc_proto
67     ${_REFLECTION}
68     ${_GRPC_GRPCPP}
69     ${_PROTOBUF_LIBPROTOBUF})
70
```



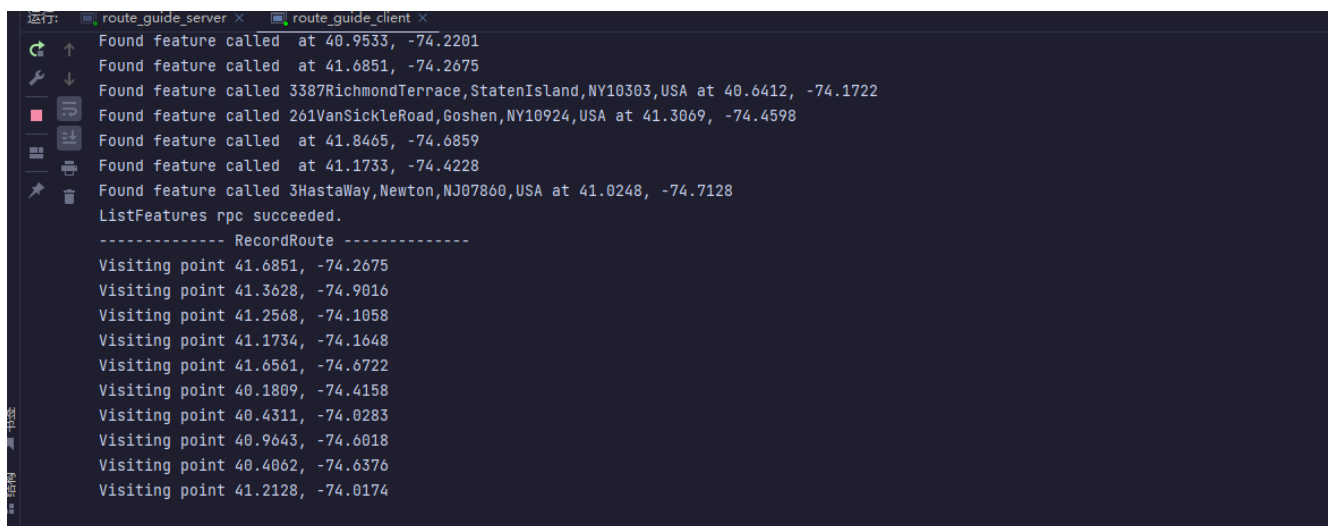
然后加载一下 cmake 构建项目，我这里使用的 make+mingw



接着如果是使用cmd构建的需要再make一下生成的makefile，我这里直接用clion一步构建了。首先构建sever，再构建client

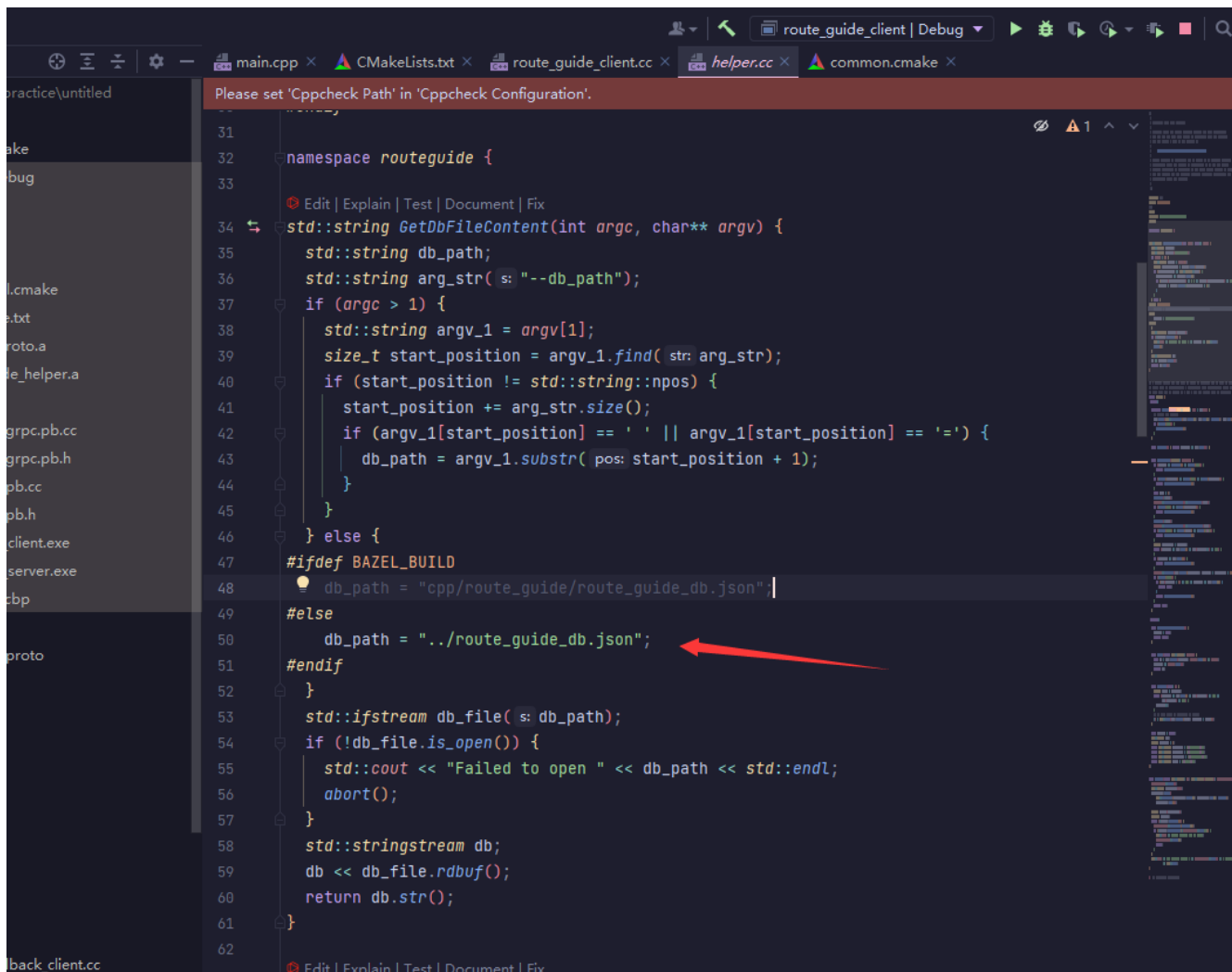


```
route_guide_dbjson 67 ${_REFLECTION}  
运行: route_guide_server x  
E:\Study\practice\untitled\cmake-build-debug\route_guide_server.exe  
DB parsed, loaded 100 features.  
Server listening on 0.0.0.0:50051
```



```
route_guide_server x route_guide_client x  
Found feature called at 40.9533, -74.2201  
Found feature called at 41.6851, -74.2675  
Found feature called 3387RichmondTerrace,StatenIsland,NY10303,USA at 40.6412, -74.1722  
Found feature called 261VanSickleRoad,Goshen,NY10924,USA at 41.3069, -74.4598  
Found feature called at 41.8465, -74.6859  
Found feature called at 41.1733, -74.4228  
Found feature called 3HastaWay,Newton,NJ07860,USA at 41.0248, -74.7128  
ListFeatures rpc succeeded.  
----- RecordRoute -----  
Visiting point 41.6851, -74.2675  
Visiting point 41.3628, -74.9016  
Visiting point 41.2568, -74.1058  
Visiting point 41.1734, -74.1648  
Visiting point 41.6561, -74.6722  
Visiting point 40.1809, -74.4158  
Visiting point 40.4311, -74.0283  
Visiting point 40.9643, -74.6018  
Visiting point 40.4062, -74.6376  
Visiting point 41.2128, -74.0174
```

如果出现了找不到json文件的报错，打开helper.cc把文件路径改一下， 可以根据自己的配置方式选择不同的选项更改



使用

主要根据这个示例来简单说一下怎么使用grpc进行远程调用

cmake

首先看一下common.cmake部分，这里面是一些通用配置。因为我们是已经安装了grpc的，所以在common.cmake里面，对我们生效的是下面这部分

他会去寻找我们安装的protoc.exe,grpc_cpp_plugin.exe以及一些依赖库，这就是之前说的建议加环境变量的原因，不用我们自己写cmakelist，他会从grpc/lib下面自动查找用于配置的cmake文件。

```
# This branch assumes that gRPC and all its dependencies are already installed
# on this system, so they can be located by find_package().

# Find Protobuf installation
# Looks for protobuf-config.cmake file installed by Protobuf's cmake installation.
set(protobuf_MODULE_COMPATIBLE TRUE)
```

```

find_package(Protobuf CONFIG REQUIRED)
message(STATUS "Using protobuf ${Protobuf_VERSION}")

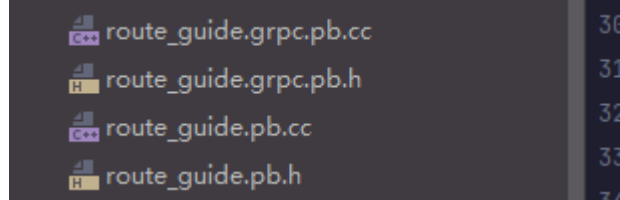
set(_PROTOBUF_LIBPROTOBUF protobuf::libprotobuf)
set(_REFLECTION gRPC::grpc++_reflection)
if (CMAKE_CROSSCOMPILING)
    find_program(_PROTOBUF_PROTOC protoc)
else ()
    set(_PROTOBUF_PROTOC $<TARGET_FILE:protobuf::protoc>)
endif ()

# Find gRPC installation
# Looks for gRPCConfig.cmake file installed by gRPC's cmake installation.
find_package(gRPC CONFIG REQUIRED)
message(STATUS "Using gRPC ${gRPC_VERSION}")

set(_GRPC_GRPCPP gRPC::grpc++)
if (CMAKE_CROSSCOMPILING)
    find_program(_GRPC_CPP_PLUGIN_EXECUTABLE grpc_cpp_plugin)
else ()
    set(_GRPC_CPP_PLUGIN_EXECUTABLE $<TARGET_FILE:gRPC::grpc_cpp_plugin>)
endif ()

```

然后是CMakeList.txt,这个部分会生成,grpc.pb.cc和grpc.pb.h是借助grpc_cpp_plugin.exe生成的



```

add_custom_command(
    OUTPUT "${rg_proto_srcs}" "${rg_proto_hdrs}" "${rg_grpc_srcs}" "${rg_grpc_hdrs}"
    COMMAND ${_PROTOBUF_PROTOC}
    ARGS --grpc_out "${CMAKE_CURRENT_BINARY_DIR}"
    --cpp_out "${CMAKE_CURRENT_BINARY_DIR}"
    -I "${rg_proto_path}"
    --plugin=protoc-gen-grpc= "${_GRPC_CPP_PLUGIN_EXECUTABLE}"
    "${rg_proto}"
    DEPENDS "${rg_proto}")

# Include generated *.pb.h files
include_directories("${CMAKE_CURRENT_BINARY_DIR}")

```

proto

定义服务

要定义一个服务,你必须在 .proto 文件中指定 service

```
service RouteGuide {  
  
}
```

然后在服务中定义 rpc 方法，指定请求的和响应类型。 gRPC根据消息的数量，将通信模式分为以下四种，在 RouteGuide 服务中都有使用：

1. 一元RPC

- **模式描述：**客户端向服务器发送一个请求，服务器处理该请求并返回一个响应。这是最基本、最常见的gRPC调用模式，类似于传统的函数调用。
- **工作流程：**
 1. 客户端构建请求消息，通过Stub（客户端存根）调用远程服务方法。
 2. 存根将请求序列化后通过网络发送到服务器。
 3. 服务器端接收到请求，反序列化后进行处理。
 4. 服务器构建响应消息，序列化后返回给客户端。
 5. 客户端接收到响应，反序列化后获取最终结果。

```
rpc GetFeature(Point) returns (Feature) {}
```

2. 服务器流RPC

- **模式描述：**客户端向服务器发送一个请求，服务器返回一个数据流作为响应。服务器可以根据客户端的请求，不断地向客户端推送数据。
- **工作流程：**
 1. 客户端构建请求并发送。
 2. 服务器接收请求并处理。
 3. 服务器根据处理逻辑，多次向客户端发送响应数据，直到没有数据可发送。
 4. 客户端在接收到流数据后，进行相应处理，直到流结束。

```
rpc ListFeatures(Rectangle) returns (stream Feature) {}
```

3. 客户端流RPC

- **模式描述：**客户端向服务器发送一个数据流作为请求，服务器处理完所有数据后返回一个单一的响应。这种模式允许客户端在短时间内将大量数据发送到服务器。
- **工作流程：**
 1. 客户端开始构建数据流，逐个向服务器发送请求数据。
 2. 服务器持续接收客户端发送的数据，进行缓存或部分处理。
 3. 客户端发送完所有数据后，通知服务器数据发送完毕。
 4. 服务器对收到的所有数据进行最终处理，构建响应并返回给客户端。

```
rpc RecordRoute(stream Point) returns (RouteSummary) {}
```

4. 双向流RPC

- **模式描述：**客户端和服务器都可以同时发送和接收数据流，双方的数据传输可以在任意时刻进行，互不干扰。

■ 工作流程：

1. 客户端开始向服务器发送数据流。
2. 服务器在接收客户端数据的同时，也可以向客户端发送数据流。
3. 双方根据各自的业务逻辑，持续地进行数据的发送和接收，直到双方都完成数据传输。

```
rpc RouteChat(stream RouteNote) returns (stream RouteNote) {}
```

我们的 .proto 文件也包含了所有请求的 protocol buffer 消息类型定义以及在服务方法中使用的响应类型-比如，下面的 Point 消息类型：

```
// Points are represented as latitude-longitude pairs in the E7 representation
// (degrees multiplied by 10**7 and rounded to the nearest integer).
// Latitudes should be in the range +/- 90 degrees and longitude should be in
// the range +/- 180 degrees (inclusive).
message Point {
    int32 latitude = 1;
    int32 longitude = 2;
}
```

接着生成cpp代码，运行类似下面的命令,修改一下文件路径就可以了，当然也可以像之前那样直接写在cmake里面

```
protoc -I ../../protos --grpc_out=. --plugin=protoc-gen-
grpc=D:\Clion\grpc\bin\grpc_cpp_plugin.exe ../../protos/route_guide.proto
protoc -I ../../protos --cpp_out=. ../../protos/route_guide.proto
```

运行这个命令可以在当前目录中生成下面的文件：

- route_guide.pb.h， 声明生成的消息类的头文件
- route_guide.pb.cc， 包含消息类的实现
- route_guide.grpc.pb.h， 声明你生成的服务类的头文件
- route_guide.grpc.pb.cc， 包含服务类的实现

存根

存根（Stub）是分布式系统和RPC（Remote Procedure Call，远程过程调用）框架中的一个重要概念，它提供了一种机制，使得本地代码能够像调用本地函数一样调用远程服务。以下从不同角度对存根进行详细阐述：

1. 基本概念

- **定义：**存根本质上是一段代码，它充当了本地应用程序与远程服务之间的代理。它接收本地应用程序的调用请求，对请求进行处理（如序列化数据），然后通过网络将请求发送到远程服务端。当远程服务处理完请求并返回响应时，存根再接收响应（反序列化数据），并将结果返回给本地应用程序。
- **作用：**存根隐藏了网络通信和远程服务调用的复杂性，让开发人员可以像调用本地函数一样调用远程服务，从而提高开发效率，使分布式系统的开发更加便捷。它就像是远程服务在本地的一个“代表”，负责与远程服务进行交互。

2. 工作原理

■ 请求处理：

- **本地调用：**当本地应用程序调用存根中的方法时，存根首先会收集调用参数。例如，在一个简单的用户登录RPC调用中，存根会收集用户名和密码等参数。
- **数据序列化：**存根将收集到的参数进行序列化，将其转换为适合在网络上传输的格式，如二进制流或JSON字符串。这一步是为了确保数据能够在不同系统之间准确传输。
- **网络传输：**存根通过网络协议（如TCP/IP）将序列化后的请求发送到远程服务所在的服务器。

■ 响应处理：

- **接收响应：**远程服务处理完请求后，将响应结果通过网络返回给存根。
- **数据反序列化：**存根接收到响应后，将其从网络传输格式反序列化为本地应用程序能够理解的数据结构。例如，将二进制流转换为对象或基本数据类型。
- **返回结果：**存根将反序列化后的结果返回给发起调用的本地应用程序代码，就好像是本地函数调用的返回值一样。

3. 生成方式

- **代码生成工具：**在许多RPC框架（如gRPC、Thrift等）中，存根代码通常由专门的代码生成工具根据服务接口定义文件（如gRPC的.proto文件、Thrift的.thrift文件）自动生成。这些工具会根据接口定义生成不同编程语言的存根代码，使得开发人员可以在不同语言环境下方便地调用远程服务。例如，使用gRPC时，通过protoc编译器结合相应的插件，可以生成C++、Java、Python等多种语言的存根代码。
- **手动编写：**在一些简单的分布式系统中，开发人员也可以手动编写存根代码。但这种方式较为繁琐，需要开发人员自行处理网络通信、数据序列化与反序列化等细节，并且容易出错。因此，在实际开发中，自动生成存根代码的方式更为常见。

4. 分类

- **客户端存根：**客户端存根位于客户端应用程序中，负责接收客户端的调用请求，并将其发送到远程服务器，
- 同时接收服务器返回的响应并返回给客户端应用程序。在上述用户登录的例子中，客户端存根运行在用户设备的应用程序中，处理与服务器的登录请求交互。
- **服务器端存根：**服务器端存根（有时也称为骨架，Skeleton）位于服务器端，
- 它接收来自网络请求，反序列化请求数据，然后调用本地实际的服务实现代码来处理请求。处理完成后，将响应结果序列化并返回给客户端。
- 服务器端存根起到了连接网络请求和本地服务实现的桥梁作用。

5. 示例

以gRPC框架为例，假设我们有一个简单的用户服务接口定义在user.proto文件中：

```
syntax = "proto3";

package user;

service UserService {
  rpc Login(LoginRequest) returns (LoginResponse);
}

message LoginRequest {
  string username = 1;
  string password = 2;
}

message LoginResponse {
  bool success = 1;
```

```
    string message = 2;
}
```

使用protoc工具生成Python客户端存根代码后，在客户端代码中可以这样调用远程服务：

```
import grpc
import user_pb2
import user_pb2_grpc

def run():
    channel = grpc.insecure_channel('localhost:50051')
    stub = user_pb2_grpc.UserServiceStub(channel)
    request = user_pb2.LoginRequest(username='testuser', password='testpass')
    response = stub.Login(request)
    print("Login response: success={}, message={}".format(response.success, response.message))

if __name__ == '__main__':
    run()
```

在上述代码中，user_pb2_grpc.UserServiceStub就是生成的客户端存根，它提供了Login方法，客户端可以像调用本地函数一样调用该方法来发起远程登录请求。存根会自动处理请求的序列化、网络传输以及响应的反序列化等操作。

服务端

RouteGuide 服务工作有两个部分：

- 实现我们服务定义的生成的服务接口：做我们的服务的实际的“工作”。
- 运行一个 gRPC 服务器，监听来自客户端的请求并返回服务的响应。

route_guide_server.cc是服务端的实现代码。现在研究它是如何工作的。

服务端有一个实现了生成的 RouteGuide::Service 接口的 RouteGuideImpl 类：

```
class RouteGuideImpl final : public RouteGuide::Service {
}
```

这个例子实现的是同步版本的RouteGuide，它提供了gRPC服务端缺省的行为。同时，也可以去实现一个异步的接口RouteGuide::AsyncService，在这个示例中存在一个回调版本实现。

RouteGuideImpl 实现了所有的服务方法。

首先看最简单的一元rpc类型 GetFeature，它从客户端拿到一个 Point 然后将对应的特性返回给数据库中的Feature。

```

    Status GetFeature(ServerContext* context, const Point* point,
Feature* feature) override {
    feature->set_name(GetFeatureName(*point, feature_list_));
    feature->mutable_location()->CopyFrom(*point);
    return Status::OK;
}

```

这个方法为 RPC 传递了一个上下文对象，包含了客户端的 Point protobuf 请求 以及一个填充响应信息的 Feature protobuf。在这个方法中，我们用适当的信息填充 Feature，然后返回OK的状态，告诉 gRPC 我们已经处理完 RPC，并且 Feature 可以返回给客户端。

现在看看更加复杂点的流式RPC。 ListFeatures 是一个服务器端的流式 RPC，因此我们需要给客户端返回多个 Feature。

```

Status ListFeatures(ServerContext* context,
                    const routeguide::Rectangle* rectangle,
                    ServerWriter<Feature>* writer) override {
    auto lo = rectangle->lo();
    auto hi = rectangle->hi();
    long left = (std::min)(lo.longitude(), hi.longitude());
    long right = (std::max)(lo.longitude(), hi.longitude());
    long top = (std::max)(lo.latitude(), hi.latitude());
    long bottom = (std::min)(lo.latitude(), hi.latitude());
    for (const Feature& f : feature_list_) {
        if (f.location().longitude() >= left &&
            f.location().longitude() <= right &&
            f.location().latitude() >= bottom && f.location().latitude() <= top) {
            writer->Write(f);
        }
    }
    return Status::OK;
}

```

我们拿到了一个请求对象(客户端期望在 Rectangle 中找到的 Feature)以及一个特殊的 ServerWriter 对象，而不是在我们的方法参数中获取简单的请求和响应对象。在方法中，根据返回的需要填充足够多的 Feature 对象，用 ServerWriter 的 Write() 方法写入。最后，和我们一元RPC 例子相同，我们返回Status::OK去告知gRPC我们已经完成了响应的写入。

客户端流方法RecordRoute，和服务端流很类似，比较有区别的地方在于，客户端流中我们拿到的是一个ServerReader而不是请求对象和单一的响应。我们使用 ServerReader 的 Read() 方法去重复的往请求对象(在这个场景下是一个 Point) 读取客户端的请求直到没有更多的消息：在每次调用后，服务器需要检查 Read() 的返回值。如果返回值为 true，流仍然存在，它就可以继续读取；如果返回值为 false，则表明消息流已经停止。

```

Status RecordRoute(ServerContext *context, ServerReader<Point> *reader,
RouteSummary *summary) override {
    Point point;
    int point_count = 0;
    int feature_count = 0;
    float distance = 0.0;
    Point previous;

    system_clock::time_point start_time = system_clock::now();
    while (reader->Read(&point)) {
        point_count++;

```



```

if (!GetFeatureName(point, feature_list_).empty()) {
    feature_count++;
}
if (point_count != 1) {
    distance += GetDistance(previous, point);
}
previous = point;
}
system_clock::time_point end_time = system_clock::now();
summary->set_point_count(point_count);
summary->set_feature_count(feature_count);
summary->set_distance(static_cast<long>(distance));
auto secs =
std::chrono::duration_cast<std::chrono::seconds>(end_time - start_time);
summary->set_elapsed_time(secs.count());

return Status::OK;
}

```

最后，让我们看看双向流RPCRouteChat()。

```

Status RouteChat(ServerContext* context,
                 ServerReaderWriter<RouteNote, RouteNote>* stream) override {
    RouteNote note;
    while (stream->Read(&note)) {
        std::unique_lock<std::mutex> lock(mu_);
        for (const RouteNote& n : received_notes_) {
            if (n.location().latitude() == note.location().latitude() &&
                n.location().longitude() == note.location().longitude()) {
                stream->Write(n);
            }
        }
        received_notes_.push_back(note);
    }

    return Status::OK;
}

```

这次我们得到的 ServerReaderWriter 对象可以用来读和写消息。这里读写的语法和我们客户端流以及服务器流方法是一样的。虽然每一端获取对方信息的顺序和写入的顺序一致，客户端和服务端都可以以任意顺序读写——流的操作是完全独立的。

实现完四种模式后，还需要定义一个启动服务的方法RunServer，实现后在main函数启动服务就行了，注意这里数据都存储在route_guide_db.json里面，可以自己修改一下helper.cc自己测试其他的数据

```

void RunServer(const std::string& db_path) {
    std::string server_address("0.0.0.0:50051");
    RouteGuideImpl service(db_path);

    ServerBuilder builder;
    builder.AddListeningPort(server_address, grpc::InsecureServerCredentials());
    builder.RegisterService(&service);
    std::unique_ptr<Server> server(builder.BuildAndStart());
    std::cout << "Server listening on " << server_address << std::endl;
    server->Wait();
}

```

通过使用ServerBuilder去构建和启动服务。为了做到这点，我们需要：

1. 创建服务实现类 RouteGuideImpl 的一个实例。
2. 创建工厂类 ServerBuilder 的一个实例。
3. 在生成器的 AddListeningPort() 方法中指定客户端请求时监听的地址和端口。
4. 用生成器注册服务实现。
5. 调用生成器的 BuildAndStart() 方法为服务创建和启动一个RPC服务器。
6. 调用服务器的 Wait() 方法实现阻塞等待，直到进程被杀死或者 Shutdown() 被调用。

客户端

为RouteGuide服务创建一个C++的客户端。route_guide_client.cc里面是完整的客户端示例代码。

为了能调用服务的方法，我们得先创建一个 存根。

首先需要为存根创建一个gRPC channel，指定我们想连接的服务器地址和端口，以及 channel 相关的参数，这里我们使用了默认的 ChannelArguments (ChannelArguments 是用于配置 gRPC 通道 (Channel) 行为的一组参数集合)

```

RouteGuideClient guide(
    grpc::CreateChannel("localhost:50051",
        grpc::InsecureChannelCredentials()));

```

以利用channel，使用从proto中生成的RouteGuide类提供的NewStub方法去创建存根。

```

RouteGuideClient(std::shared_ptr<Channel> channel, const std::string& db)
    : stub_(RouteGuide::NewStub(channel)) {
    routeguide::ParseDb(db, &feature_list_);
}

```

然后就是如何对四种模式进行调用：

调用简单 RPC GetFeature 和调用一个本地方法很类似。

```

void GetFeature() {
    Point point;
    Feature feature;
    point = MakePoint(409146138, -746188906);
}

```

```

    GetOneFeature(point, &feature);
    point = MakePoint(0, 0);
    GetOneFeature(point, &feature);
}

private:
bool GetOneFeature(const Point& point, Feature* feature) {
    ClientContext context;
    Status status = stub_->GetFeature(&context, point, feature);
    if (!status.ok()) {
        std::cout << "GetFeature rpc failed." << std::endl;
        return false;
    }
    if (!feature->has_location()) {
        std::cout << "Server returns incomplete feature." << std::endl;
        return false;
    }
    if (feature->name().empty()) {
        std::cout << "Found no feature at "
                    << feature->location().latitude() / kCoordFactor_ << ", "
                    << feature->location().longitude() / kCoordFactor_ << std::endl;
    } else {
        std::cout << "Found feature called " << feature->name() << " at "
                    << feature->location().latitude() / kCoordFactor_ << ", "
                    << feature->location().longitude() / kCoordFactor_ << std::endl;
    }
    return true;
}
}

```

这里创建并且填充了一个请求的 protocol buffer 对象（例子中为 Point），同时为了服务器填写创建了一个响应 protocol buffer 对象。为了调用还创建了一个 ClientContext 对象，可以随意的设置该对象上的配置的值，比如期限，虽然现在会使用缺省的设置。注意，不能在不同的调用间重复使用这个对象。最后，在存根上调用这个方法，将其传给上下文，请求以及响应。如果方法的返回是OK，那么我们就可以从服务器的响应对象中读取响应信息。

```

std::cout << "Found feature called " << feature->name() << " at "
            << feature->location().latitude()/kCoordFactor_ << ", "
            << feature->location().longitude()/kCoordFactor_ << std::endl;

```

然后是流RPC，其实和服务端的实现很类似，下面是服务器流

```

void ListFeatures() {
    routeguide::Rectangle rect;
    Feature feature;
    ClientContext context;

    rect.mutable_lo()->set_latitude(400000000);
    rect.mutable_lo()->set_longitude(-750000000);
    rect.mutable_hi()->set_latitude(420000000);
    rect.mutable_hi()->set_longitude(-730000000);
    std::cout << "Looking for features between 40, -75 and 42, -73"
              << std::endl;

    std::unique_ptr<ClientReader<Feature>> reader(
        stub_->ListFeatures(&context, rect));
}

```

```

while (reader->Read(&feature)) {
    std::cout << "Found feature called " << feature.name() << " at "
               << feature.location().latitude() / kCoordFactor_ << ", "
               << feature.location().longitude() / kCoordFactor_ << std::endl;
}
Status status = reader->Finish();
if (status.ok()) {
    std::cout << "ListFeatures rpc succeeded." << std::endl;
} else {
    std::cout << "ListFeatures rpc failed." << std::endl;
}
}

```

将上下文传给方法并且请求，得到 ClientReader 返回对象，而不是将上下文，请求和响应传给方法。客户端可以使用 ClientReader 去读取服务器的响应。接着使用 ClientReader 的 Read() 反复读取服务器的响应到一个响应 protocol buffer 对象 (在这个例子中是一个 Feature)，直到没有更多的消息：客户端需要去检查每次调用完 Read() 方法的返回值。如果返回值为 true，流依然存在并且可以持续读取；如果是 false，说明消息流已经结束。最后，在流上调用 Finish() 方法结束调用并获取我们 RPC 的状态。

客户端的流方法 RecordRoute 的使用很相似，有区别的点在于将一个上下文和响应对象传给方法，拿到一个 ClientWriter 返回。

```

void RecordRoute() {
    Point point;
    RouteSummary stats;
    ClientContext context;
    const int kPoints = 10;
    unsigned seed = std::chrono::system_clock::now().time_since_epoch().count();

    std::default_random_engine generator(seed);
    std::uniform_int_distribution<int> feature_distribution(
        0, feature_list_.size() - 1);
    std::uniform_int_distribution<int> delay_distribution(500, 1500);

    std::unique_ptr<ClientWriter<Point> > writer(
        stub->RecordRoute(&context, &stats));
    for (int i = 0; i < kPoints; i++) {
        const Feature& f = feature_list_[feature_distribution(generator)];
        std::cout << "Visiting point " << f.location().latitude() / kCoordFactor_
                  << ", " << f.location().longitude() / kCoordFactor_
                  << std::endl;
        if (!writer->Write(f.location())) {
            // Broken stream.
            break;
        }
        std::this_thread::sleep_for(
            std::chrono::milliseconds(delay_distribution(generator)));
    }
    writer->WritesDone();
    Status status = writer->Finish();
    if (status.ok()) {
        std::cout << "Finished trip with " << stats.point_count() << " points\n"
                  << "Passed " << stats.feature_count() << " features\n"
                  << "Travelled " << stats.distance() << " meters\n"
                  << "It took " << stats.elapsed_time() << " seconds"
    }
}

```

```

        << std::endl;
    } else {
        std::cout << "RecordRoute rpc failed." << std::endl;
    }
}

```

用 Write() 将客户端请求写入到流的动作完成，接着在流上调用 WritesDone() 通知 gRPC 我们已经完成写入，然后调用 Finish() 完成调用同时拿到 RPC 的状态。如果状态是 OK，最初传给 RecordRoute() 的响应对象会跟着服务器的响应被填充。

最后是双向流式 RPC RouteChat()。在这种场景下，我们将上下文传给一个方法，拿到一个可以用来读写消息的 ClientReaderWriter 的返回

```

std::shared_ptr<ClientReaderWriter<RouteNote, RouteNote> > stream(
    stub->RouteChat(&context));

```

这里读写的语法和我们客户端流以及服务器端流方法没有任何区别。虽然每一方都能按照写入时的顺序拿到另一方的消息，客户端和服务端都可以以任意顺序读写——流操作起来是完全独立的。

```

void RouteChat() {
    ClientContext context;

    std::shared_ptr<ClientReaderWriter<RouteNote, RouteNote> > stream(
        stub->RouteChat(&context));

    std::thread writer([stream]() {
        std::vector<RouteNote> notes{MakeRouteNote("First message", 0, 0),
                                     MakeRouteNote("Second message", 0, 1),
                                     MakeRouteNote("Third message", 1, 0),
                                     MakeRouteNote("Fourth message", 0, 0)};
        for (const RouteNote& note : notes) {
            std::cout << "Sending message " << note.message() << " at "
                      << note.location().latitude() << ", "
                      << note.location().longitude() << std::endl;
            stream->Write(note);
        }
        stream->WritesDone();
    });

    RouteNote server_note;
    while (stream->Read(&server_note)) {
        std::cout << "Got message " << server_note.message() << " at "
                  << server_note.location().latitude() << ", "
                  << server_note.location().longitude() << std::endl;
    }
    writer.join();
    Status status = stream->Finish();
    if (!status.ok()) {
        std::cout << "RouteChat rpc failed." << std::endl;
    }
}

```

API

Cpp的gRPC的API文档地址: <https://grpc.github.io/grpc/cpp/>

gRPC其他教程: <https://grpc.io/docs/languages/cpp/>